

HelixOTA

HelixOTA

सार्वभौमिक, पूर्णतः पृथक्कृत ओवर-द-एयर अपडेट — डिज़ाइन द्वारा शून्य-ब्रिक।

सारांश

Helix OTA एक सार्वभौमिक, गहराई से पृथक्कृत ओवर-द-एयर अपडेट प्रणाली है: एक Go नियंत्रण प्लेन के साथ-साथ प्रति-ओएस क्लाइंट एजेंट, जिसे एकल बोर्ड से लेकर लाखों उपकरणों के समूह तक सुरक्षित, चरणबद्ध फर्मवेयर/ऐप अपडेट प्रदान करने के लिए डिज़ाइन किया गया है। इसका पहला लक्ष्य ऑरेंज पाई 5 मैक्स पर एंड्रॉयड 15 है।

संक्षिप्त विवरण

Helix OTA एक सार्वभौमिक ओवर-द-एयर अपडेट प्रणाली है — एक Go नियंत्रण प्लेन के साथ-साथ प्रति-ओएस क्लाइंट एजेंट — जिसे शून्य सिस्टम भ्रष्टाचार, सत्यापित अपलोड और विस्तृत चरणबद्ध रोलआउट के लिए तैयार किया गया है। इसका पहला लक्ष्य ऑरेंज पाई 5 मैक्स पर एंड्रॉयड 15 है, जबकि Linux/विंडोज़ एडेप्टर की योजना बनाई गई है।

विस्तृत विवरण

Helix OTA एक सार्वभौमिक, सामान्य, गहराई से पृथक्कृत ओवर-द-एयर (OTA) अपडेट प्रणाली है, जिसे एक अटल वचन के लिए बनाया गया है: अपडेट कभी भी किसी काम कर रहे उपकरण को ईट में नहीं बदल सकता। इसमें एक Go सर्वर नियंत्रण प्लेन, प्रति-ओएस क्लाइंट एजेंट, और एक प्रबंधन डैशबोर्ड शामिल है, और इसे इस तरह से डिज़ाइन किया गया है कि यह प्लगइन ओएस एडेप्टर के माध्यम से किसी भी ऑपरेटिंग सिस्टम में एम्बेड किया जा सके, बजाय इसके कि हर प्लेटफ़ॉर्म के लिए इसे नए सिरे से बनाया जाए। पहला वितरण लक्ष्य ऑरेंज पाई 5 मैक्स पर एंड्रॉयड 15 (सभी वेरिएंट) है, जहाँ बिल्ड पाइपलाइन फ्लैशिंग इमेज के साथ-साथ एक सत्यापित OTA .zip और अनिवार्य हैश फ़ाइलें उत्पन्न करती है, ताकि कोई भी आर्टिफैक्ट बिना सत्यापन योग्य फ़िगरप्रिंट के उपकरण तक न पहुँचे; Linux, विंडोज़ और अन्य ओएस इसी एडेप्टर सीम के पीछे रोडमैप पर हैं, जिन्हें केवल एडेप्टर की आवश्यकता है — न कि पुनर्लेखन की।

इस डिज़ाइन को ऑपरेटर द्वारा निर्धारित कठोर गारंटियों के इर्द-गिर्द व्यवस्थित किया गया है, जिन्हें गैर-परक्राम्य वास्तुशिल्पीय स्थिरांक के रूप में माना जाता है: शून्य सिस्टम भ्रष्टाचार, हर आर्टिफैक्ट का अनिवार्य सत्यापन तैनाती से पहले, विस्तृत रोलआउट (एक

साथ या 5/10/30...100% चरणों में रुकावट और आगे बढ़ने की सुविधा के साथ), समूह की पूर्ण दृश्यता, और एकल बोर्ड से लेकर क्षेत्र में लाखों उपकरणों तक रेखिक स्केल। लॉकड आर्किटेक्चर डिवाइस-साइड नेटिव एंड्रॉयड A/B अपडेट — AOSP update_engine के साथ AVB/dm-verity और स्वचालित बूट-फेलियर रोलबैक — को एक कस्टम, पृथक्कृत Go नियंत्रण प्लेन के साथ जोड़ता है, ताकि सुरक्षा सिलिकॉन-सन्निकट बूट पथ और सर्वर में निहित हो, न कि किसी एक नाज़ुक परत में। दो सीमाओं को जानबूझकर अलग करने योग्य रखा गया है: एक ओएस-एडाप्टर सीम जो वास्तविक सार्वभौमिकता का वादा करती है, और एक रोलआउट-इंजन सीम जो चरणबद्ध अभियानों को ओएस-अज्ञेय बनाती है। पूरी प्रणाली को छह सार्वजनिक, स्वतंत्र रूप से संस्करणित ota-* सबमॉड्यूल में विभाजित किया गया है — एक मोनोलिथ के बजाय पुनः उपयोग योग्य बिल्डिंग ब्लॉक।

Helix OTA वर्तमान में स्पेसिफिकेशन/शोध और परीक्षण कवरेज के निर्माण चरण में है; रिपॉजिटरी में आधिकारिक डिज़ाइन संग्रह, दस्तावेज़ निर्यात पाइपलाइन और सबमॉड्यूल स्कैफोल्डिंग शामिल है, और यह स्पष्ट रूप से — अपने एंटी-ब्लफ़ गवर्नेंस के अनुसार — कहा गया है कि अभी तक कोई पूर्ण उत्पादन सर्वर और एजेंट मौजूद नहीं है। आज जो उपलब्ध है, वह ब्लूप्रिंट और उसका स्कैफोल्डिंग है, जिसे ईमानदारी से इसी रूप में लेबल किया गया है।

सामग्री

हमने इसे क्यों बनाया

OTA आमतौर पर हर डिवाइस और हर ऑपरेटिंग सिस्टम के लिए नए सिरे से तैयार किया जाता है, और एक खराब अपडेट पूरे बेड़े को बेकार कर सकता है। Helix OTA को एक सार्वभौमिक, सुरक्षा-प्राथमिकता वाला अपडेट सिस्टम के रूप में बनाया गया है जिसे कोई भी ऑपरेटिंग सिस्टम एडाप्टर के माध्यम से अपना सकता है, जिसमें रोलबैक और सत्यापन की गारंटी आर्किटेक्चर में ही शामिल है, न कि बाद में जोड़ी गई सुविधाओं के रूप में।

यह क्यों गेम-चेंजर है

यह “कभी भी डिवाइस को बेकार न करें” और “धीरे-धीरे और अवलोकन योग्य तरीके से रोल आउट करें” को ऐसी सर्वोत्तम-प्रयास सुविधाओं के रूप में नहीं मानता जिन्हें आप लोड के तहत टिके रहने की उम्मीद करते हैं—बल्कि ये बूट पथ और कंट्रोल प्लेन दोनों में ही आर्किटेक्चरल स्थिरांक के रूप में शामिल हैं। और रोलआउट इंजन और ऑपरेटिंग सिस्टम लेयर को स्वैपेबल सीम के रूप में बनाकर, न कि कठोर पूर्वधारणाओं के रूप में, वही कंट्रोल प्लेन आज एंड्रॉयड को संचालित कर सकता है और बाद में केवल एक एडाप्टर जोड़कर अन्य ऑपरेटिंग सिस्टम को संचालित करने के लिए तैयार रह सकता है—बिना फोर्क, बिना रीराइट, बिना उन सुरक्षा गारंटियों को फिर से ईजाद किए जिन्हें आप पहले से ही भरोसेमंद मानते हैं।

क्या नया है

- दो अलग करने योग्य सीम — एक OS-एडाप्टर सीम और एक OS-अज्ञेयवादी रोलआउट इंजन — जो “सार्वभौमिक” को एक मार्केटिंग शब्द से कोडबेस की संरचनात्मक विशेषता में बदल देता है।

- **गहराई से रक्षा:** डिवाइस-साइड नेटिव A/B (update_engine) + AVB/dm-verity + स्वचालित बूट-फेलियर रोलबैक, सर्वर-साइड आर्टिफैक्ट सत्यापन के ऊपर परतबद्ध — एक अपडेट को स्थायी होने से पहले कई स्वतंत्र गेट से गुजरना पड़ता है।
- **कैटलॉग-प्रथम, डिक्पल्ड डिज़ाइन** जिसमें छह पुनःप्रयोगी, स्वतंत्र रूप से वर्जन किए गए ota-* सबमॉड्यूल शामिल हैं जिन्हें आप अपनी पसंद के अनुसार चुन सकते हैं, बजाय एक मोनोलिथ को निगलने के।
- **HTTP/3 (QUIC) प्राथमिक ट्रांसपोर्ट** जिसमें स्वचालित HTTP/2 फॉलबैक और नेगोशिएटेड Brotli/gzip कम्प्रेशन है — आधुनिक, कम-विलंबता डिलीवरी जो विफल होने के बजाय धीरे-धीरे डिग्रेड होती है।
- **धोखाधड़ी-विरोधी इंजीनियरिंग:** डिज़ाइन और स्थिति को स्पष्ट रूप से स्पेक-फेज़ के रूप में चिह्नित किया गया है, और जो कुछ भी बनाया नहीं गया है उसे कभी भी शिफ्ट के रूप में दावा नहीं किया जाता — ईमानदारी को इंजीनियरिंग मूल्य के रूप में लागू किया गया है, न कि फुटनोट में एक अस्वीकरण के रूप में।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **यह गारंटी देना कि एक खराब अपडेट कभी भी डिवाइस को बेकार न करे** — OTA का सबसे कठिन वादा। इसका समाधान डिवाइस-साइड नेटिव एंड्रॉयड A/B को अनिवार्य बनाकर किया गया: update_engine निष्क्रिय स्लॉट में लिखता है जबकि सक्रिय स्लॉट चलता रहता है, AVB/dm-verity बूट चेन को क्रिप्टोग्राफिक रूप से सत्यापित करता है, और अगर नया स्लॉट बूट होने में विफल रहता है तो डिवाइस स्वचालित रूप से रोलबैक कर लेता है — यह सब अनिवार्य प्री-डिप्लॉय आर्टिफैक्ट सत्यापन द्वारा समर्थित है ताकि एक भ्रष्ट पेलोड सर्वर से बाहर निकलने से पहले ही पकड़ लिया जाए।
- **एक सिस्टम, कई ऑपरेटिंग सिस्टम** — इसका समाधान कोर में एंड्रॉयड की पूर्वधारणाओं को शामिल न करके किया गया। एक प्लगेबल OS-एडाप्टर सीम प्लेटफॉर्म-विशिष्टताओं को अलग करता है, और एक OS-अज्ञेयवादी रोलआउट इंजन सीम अभियान तर्क को पोर्टेबल रखता है, प्रत्येक को एक अलग सबमॉड्यूल के रूप में रखा गया है ताकि एक नया OS पूरे सिस्टम में सर्जरी किए बिना जोड़ा जा सके।
- **चरणबद्ध, रोकने योग्य रोलआउट** — इसका समाधान एक समर्पित रोलआउट इंजन द्वारा किया गया जो प्रतिशत कोहोर्ट्स में सोचता है, जिसमें सफलता/त्रुटि थ्रेशोल्ड और स्पष्ट रोक/आगे बढ़ाने का नियंत्रण होता है, जानबूझकर HTTP कपलिंग से मुक्त रखा गया ताकि वही इंजन ट्रांसपोर्ट से स्वतंत्र अभियानों को संचालित कर सके।

सामग्री

तकनीकी स्टैक

- **Go + Gin** — समवर्ती मॉडल और हल्के परिनियोजन पदचिह्न के लिए चुना गया; नियंत्रण तल, रोलआउट इंजन और आर्टिफैक्ट सत्यापनकर्ताओं को शक्ति प्रदान करता है, जो REST /api/v1 प्राथमिक इंटरफ़ेस को उजागर करता है।
- **Kotlin/KMP** — इस प्रकार चुना गया ताकि डिवाइस पर मौजूद एंड्रॉयड OTA एजेंट विभिन्न लक्ष्यों पर तर्क साझा कर सके, डिवाइस के संपूर्ण लूप—पोल/डाउनलोड/सत्यापन/लागू/रिपोर्ट—का स्वामित्व रखता है।
- **HTTP/3 (QUIC) → HTTP/2** — QUIC को कम विलंबता और अस्थिर मोबाइल कनेक्शनों पर लचीले वितरण के लिए प्राथमिक परिवहन के रूप में चुना गया, जिसमें स्वचालित HTTP/2 फॉलबैक है ताकि कोई डिवाइस अटके नहीं; **Brotli/gzip** अनुरोध के अनुसार बातचीत द्वारा पेलोड को संक्षिप्त करता है।

- **PostgreSQL** — डिवाइस रजिस्ट्री, अभियानों और टेलीमेट्री में संबंधपरक अखंडता के लिए चुना गया, जहाँ कच्ची लेखन गति की तुलना में बेड़े की स्थिति की शुद्धता अधिक महत्वपूर्ण है।
- **MinIO / S3** — आर्टिफैक्ट ब्लॉब स्टोर के रूप में चुना गया ताकि बड़े फर्मवेयर इमेज वस्तु भंडारण में रहें, जो संबंधपरक परत से अलग हो।
- **AOSP update engine + AVB/dm-verity + boot_control** — एंड्रॉइड के स्वयं के युद्ध-परीक्षित वर्चुअल A/B और सत्यापित-बूट तंत्र का पुनः उपयोग करना एक नया अपडेटर बनाने से अधिक सुरक्षित है; इसका उपयोग डिवाइस पर स्टॉट स्वैप और क्रिप्टोग्राफिक बूट सत्यापन के लिए किया जाता है।
- **React** — प्रबंधन डैशबोर्ड के लिए चुना गया जहाँ ऑपरेटर लॉग इन करते हैं, आर्टिफैक्ट अपलोड करते हैं, रोलआउट संचालित करते हैं और एक ही स्थान पर बेड़े के स्वास्थ्य की निगरानी करते हैं।
- **OpenTelemetry + Prometheus/Grafana** — विक्रेता-तटस्थ उपकरणिकरण के लिए चुना गया; इसका उपयोग रोलआउट के हर चरण को मेट्रिक्स और डैशबोर्ड में दृश्यमान बनाने के लिए किया जाता है, न कि अनुमान पर आधारित।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** विकासाधीन। परियोजना की स्वयं की भ्रामकता-विरोधी शासन नीति के अनुसार, अभी तक कोई कार्यशील उत्पादन सर्वर या एजेंट नहीं है — यह एक विनिर्देश/शोध और परीक्षण-कवरेज निर्माण चरण है। रिपॉजिटरी में आधिकारिक डिज़ाइन संग्रह, दस्तावेज़ निर्यात पाइपलाइन और सबमॉड्यूल संरचना मौजूद है।
- छह सार्वजनिक पुनः उपयोगी सबमॉड्यूल (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) github.com/HelixDevelopment/ के अंतर्गत मौजूद हैं।
- रिपॉजिटरी में परीक्षण-कवरेज और विलंबता के आँकड़े परियोजना के स्वयं के प्रगतिशील रिकॉर्ड हैं, स्वतंत्र रूप से पुष्टि नहीं की गई है। README में उद्धृत HelixConstitution खंड संख्याएँ असत्यापित हैं।
- लाइसेंस: **Apache-2.0**।

प्राथमिकता स्तर: Helix-प्राथमिक।